

Training Deep Neural Networks

Overview

Most deep learning failures are training failures, not architecture failures. This chapter focuses on practical training mechanics that separate unstable experiments from production-credible models.

Core objectives:

- converge reliably,
- generalize beyond training data,
- diagnose failure modes early,
- maintain reproducible workflows.

Optimization Landscape and Training Dynamics

Deep networks optimize non-convex objectives. In practice, useful minima are common, but path to them depends heavily on optimization setup.

Important dynamics:

- gradient scale across layers,
- curvature and noisy mini-batch gradients,
- learning-rate schedule interactions,
- batch size vs convergence behavior.

Practical interpretation:

- training is a control problem: tune update dynamics, not only architecture depth.

Initialization and Normalization

Initialization

Good initialization preserves signal variance across layers.

Common patterns:

- **Xavier/Glorot** for tanh/sigmoid families,
- **He/Kaiming** for ReLU families.

Normalization

- **BatchNorm**: stabilizes activations and allows larger learning rates.
- **LayerNorm**: common in transformers and sequence models.

Benefits:

- faster convergence,
- smoother optimization,
- mild regularization effects.

Regularization and Generalization

Key regularization tools:

- **Weight decay** (L2 penalty),
- **Dropout**,
- **Data augmentation**,
- **Early stopping**,
- **Label smoothing** (classification contexts).

Generalization strategy:

- combine multiple moderate regularizers rather than one aggressive control.

Optimizers and Learning-Rate Strategy

Common optimizers:

- **SGD + momentum**: strong generalization, often slower to tune.
- **Adam/AdamW**: fast convergence, robust defaults for many workloads.

Learning-rate schedules:

- step decay,
- cosine annealing,
- one-cycle policy,
- warmup (especially for transformers).

Rule of thumb:

- learning rate dominates training behavior; tune it before deeper architecture changes.

Instrumentation and Training Diagnostics

Track at minimum:

- train loss / validation loss,
- primary validation metric,
- learning rate over steps,
- gradient norm,
- throughput and epoch time.

Diagnostics patterns:

- train down, val flat -> undercapacity or data mismatch,
- train down, val up -> overfitting,
- train noisy/diverging -> lr too high or unstable gradients.

Failure Modes and Recovery Playbook

Underfitting

Symptoms:

- both train and validation scores poor.

Actions:

- increase capacity,
- train longer,
- improve features/data quality,
- reduce excessive regularization.

Overfitting

Symptoms:

- train strong, validation weak.

Actions:

- stronger regularization,
- augmentation,

- earlier stopping,
- simpler model.

Instability

Symptoms:

- oscillating or exploding loss.

Actions:

- lower learning rate,
- gradient clipping,
- better initialization,
- check data normalization.

Business and Engineering Context

Training quality directly affects:

- deployment confidence,
- maintenance cost,
- retraining frequency,
- stakeholder trust in model behavior.

Teams that treat training as systematic engineering outperform ad-hoc tuning teams over time.

Project Prompts & Practice Exercises

1. Train an MLP on Fashion-MNIST with 3 optimizer variants and compare convergence/accuracy.
2. Run ablations: no dropout vs dropout; fixed LR vs cosine schedule.
3. Build a “training incident runbook” for exploding gradients and overfitting scenarios.
4. Create a reproducibility checklist and verify two reruns produce similar metrics.

Interview Questions & Answers

1. **Q:** What is the first hyperparameter you tune for deep networks?
A: Learning rate.
2. **Q:** Adam vs SGD with momentum?
A: Adam converges fast and is easy to use; SGD can generalize strongly with careful schedules.
3. **Q:** Why use learning-rate warmup?
A: It avoids unstable early updates in sensitive architectures.
4. **Q:** What does weight decay do?
A: Penalizes large weights to improve generalization.
5. **Q:** Why can dropout help?
A: It reduces co-adaptation and acts as stochastic regularization.
6. **Q:** What is early stopping?
A: Stopping training when validation no longer improves.
7. **Q:** How do you detect overfitting quickly?
A: Monitor widening train-vs-validation gap across epochs.
8. **Q:** Why is normalization useful in deep nets?
A: Stabilizes optimization and supports deeper effective training.
9. **Q:** What is gradient clipping?
A: Limiting gradient magnitude to prevent unstable large updates.
10. **Q:** Why track gradient norms?
A: They expose vanishing/exploding behavior early.

11. **Q:** What does a good experiment log contain?
A: Config, seed, code version, dataset version, metrics, and artifacts.
12. **Q:** How do you make training reproducible?
A: Seed control, deterministic settings where feasible, pinned deps, versioned configs.
13. **Q:** What is label smoothing intuition?
A: Prevents overconfident predictions and can improve calibration.
14. **Q:** When should you increase batch size?
A: When compute allows and gradient noise is too high, but validate generalization impact.
15. **Q:** What if validation plateaus early?
A: Revisit data pipeline, objective mismatch, LR schedule, and model capacity.

Bridge to Lesson 4.3

With training mechanics established, the next chapter applies these principles to computer vision with CNN architectures, transfer learning, and practical image pipeline decisions.